

YOLO 차선 주행 : 현재 구현과 다음 단계

- Pinky2 · 카메라 기반 local lane following · Nav2 미사용
 - 픽셀 → 로봇 좌표 → 중앙 경로 → Pure Pursuit → /cmd_vel
- local_map · 회전 /S 자 · 실험 결과 · AprilTag/ 관제 확장
- 2026.10.06 코드 및 이번 실주행 기록 기준

핵심 답변 : local_map 을 활용하고 있나 ?

- 관측 차선 저장과 /lane_autonomy/local_map 표시 : 구현됨
- 최근 10 초 이력 + 로봇 기준 전후방 · 좌우 $\pm 1\text{m}$ 공간 기록
- 현재 semantic=true: 저장 맵으로 경로를 복구하지 않음
- 기존 모드 : 별도 LocalLaneMap/ 이력을 좌우 연관 · 코너 보조에 사용

전체 역할 분리 : PC 와 로봇

Pinky2 카메라

640×480 / 15fps

cam_stream.py

이미지 + 캡처 시각

PC 계산

YOLO segmentation

흰 픽셀 · flow 보충

투영 · 경로 · 목표점

Pinky2 제어

응답 /odom 신선도 확인

Pure Pursuit / 20Hz

/cmd_vel 발행

영상 · 계산 요청 → SSH 터널 /HTTP → PC → 계획 응답 → 로봇

현재 PC↔Pinky2 추론 전송은 토픽 도메인 브릿지가 아님

YOLO 가 하는 일과 하지 않는 일

- 모델 출력 : 차선과 정지선의 segmentation polygon/mask
- 프로토콜 기준 class 1=lane, class 0=crossline
- YOLO 는 거리 · 중앙 경로 · 속도 명령을 직접 출력하지 않음
- 좌우 분류 · 기하 변환 · 조향은 검출 이후 코드가 담당

모델 · 추론 설정과 성능 해석

- 현재 모델 : yolo_runs/segment/train/weights/best.pt
- 기본 imgsz=320, confidence=0.55, IoU=0.70
- 이번 시험은 PC CPU 추론 : 로그에서 약 0.2 초 / 프레임
- 처리 시간 ≠ 입력 나이 : 전송 · 대기 · 계획 시간도 포함

YOLO + 흰 픽셀 보충은 왜 쓰나 ?

- YOLO 가 승인한 차선에 대응하는 현재 흰 영역을 보충
- optical flow 로 이전 승인 영역을 현재 프레임에 대응
 - 흰색만 보고 차선을 새로 만드는 cold start 는 하지 않음
- semantic 유효 10 초 , flow 프레임 간격 제한 0.8 초

픽셀을 base_link 좌표로 변환

- intrinsic K + distortion: 픽셀을 광선 방향으로 변환
- extrinsic R,t: upright optical → base_link 변환
 - 광선과 ground_plane_z_m 바닥 평면의 교점 계산
- x= 전방 , y= 좌측 (+), 단위 m / BEV 생성 불필요

Ground-plane projection 수식

- $q = \text{undistort}(u,v; K,D)$, $r_camera = [q_x, q_y, 1]$
- $r_base = R \cdot r_camera$, $camera_origin = t$
- $\lambda = (z_ground - t_z) / r_base_z$
- $P_base = t + \lambda \cdot r_base \rightarrow (x,y)$
 - 바닥 평면을 가정하므로 벽 · 큐브 면 픽셀에는 적용 불가

URDF 보정의 한계 : 좌표는 나와도 오차가 남는다

- URDF 는 기계 링크 위치 · 회전을 제공
 - 실제 optical 장착 방향은 별도 가정과 검증이 필요
 - 현재 경고 : mounting UNVERIFIED / nominal mounting
 - 근 · 원거리 실측 오차가 조향 · 회전 시작점에 전달됨

현재 좌우 경계 선택 : 무엇을 기준으로 하나 ?

- 연결 곡선의 near prefix 를 로봇 좌표로 평가
- median $y > 0$ 이면 LEFT, $y < 0$ 이면 RIGHT
 - 너무 중앙이거나 시작 전방 거리가 먼 후보는 제외
 - 각 side 후보 중 가상 중앙 시작점이 가까운 경계 선택

한쪽 차선만 보일 때 : tangent/normal offset

- $t = \text{선의 진행 방향}$ / $n = [-t_y, t_x]$
 - 왼쪽 경계 : 중심 = 경계 - $\text{lane_width}/2 \cdot n$
 - 오른쪽 경계 : 중심 = 경계 + $\text{lane_width}/2 \cdot n$
- 현재 semantic 폭은 설정값 0.154m 를 사용

양쪽 차선이 보일 때도 알아둘 차이

- 기존 모드 : 양 경계 대응 · 폭 추정 · 중앙 경로 생성
- 현재 semantic: 좌우 후보를 찾되 한 경계의 offset 경로 선택
- 선택된 경로를 쓰며 항상 두 선 평균을 내는 것은 아님
- 따라서 두 선 검출 수와 중앙 정확도는 별도 검증 필요

연결 골격 + miter 는 무엇인가 ?

- 검출 마스크를 연결된 바닥 곡선으로 변환
- RDP 로 작은 지터를 줄이고 선분마다 normal offset
- 꼭짓점에서 offset 선분을 교차시켜 모서리 연결
- x 로 재정렬하지 않아 90 도 횡방향 구간을 보존

lookahead: 연결 경로를 따라 목표점 선택

- 현재 semantic launch: lookahead 0.16m
- 로봇에서 가장 가까운 경로 station 이후를 사용
- 누적 호 길이로 전진 $\rightarrow$ x 감소만으로 경로를 버리지 않음
- 관측 경로가 짧으면 마지막 관측 범위까지 목표 조정

조향은 PID 인가 ? 현재는 Pure Pursuit

- $\text{target}=(x,y)$, $Ld^2=x^2+y^2$, $\kappa=2y/Ld^2$
- $\text{angular_velocity} = \text{linear_velocity} \cdot \kappa$
 - 곡률 · 목표 거리 · 단일 경계 여부에 따라 속도 제한
- semantic target $x < 4\text{cm}$ 이면 전진 0 + 각도 제한 회전

현재와 기존 : 코너 처리 방식 비교

- semantic=true: 연결 경로 · 호 길이 target · 근거리 횡방향 회전
- semantic=false: MetricLaneTracker + CornerPolicy
 - 기존 : 코너 확인 → ENTRY → BRAKE → PIVOT_TURN → 재획득
 - 새 모드는 기존 staged 코너 / 맵 복구 분기를 우회

S 자에서 왜 잘 보이는데 멈추나 ?

- 검출 → 바닥 투영 → 연결 경계 → offset → target 중 실패
- 가로선 · 분리된 mask · 출구 유실이 경로 연결을 깨뜨림
- offset 이 반전 cusp 를 만들면 중앙 경로를 거부
- YOLO/flow 가 함께 사라지면 현재 모드에 맵 복구가 없음

기억은 세 종류 : 서로 구분해야 한다

영상 대응 기억

semantic 승인 / 10 초
현재 흰 픽셀 · flow
새 모드에서도 활용

진단 local_map

최근 10 초 차선
 $\pm 1\text{m}$ 공간 기록
저장 · 그림 , 제어 아님

기존 제어 이력

LaneHistory 좌우 대응
CornerPolicy
LocalLaneMap
기존 모드 복구 보조

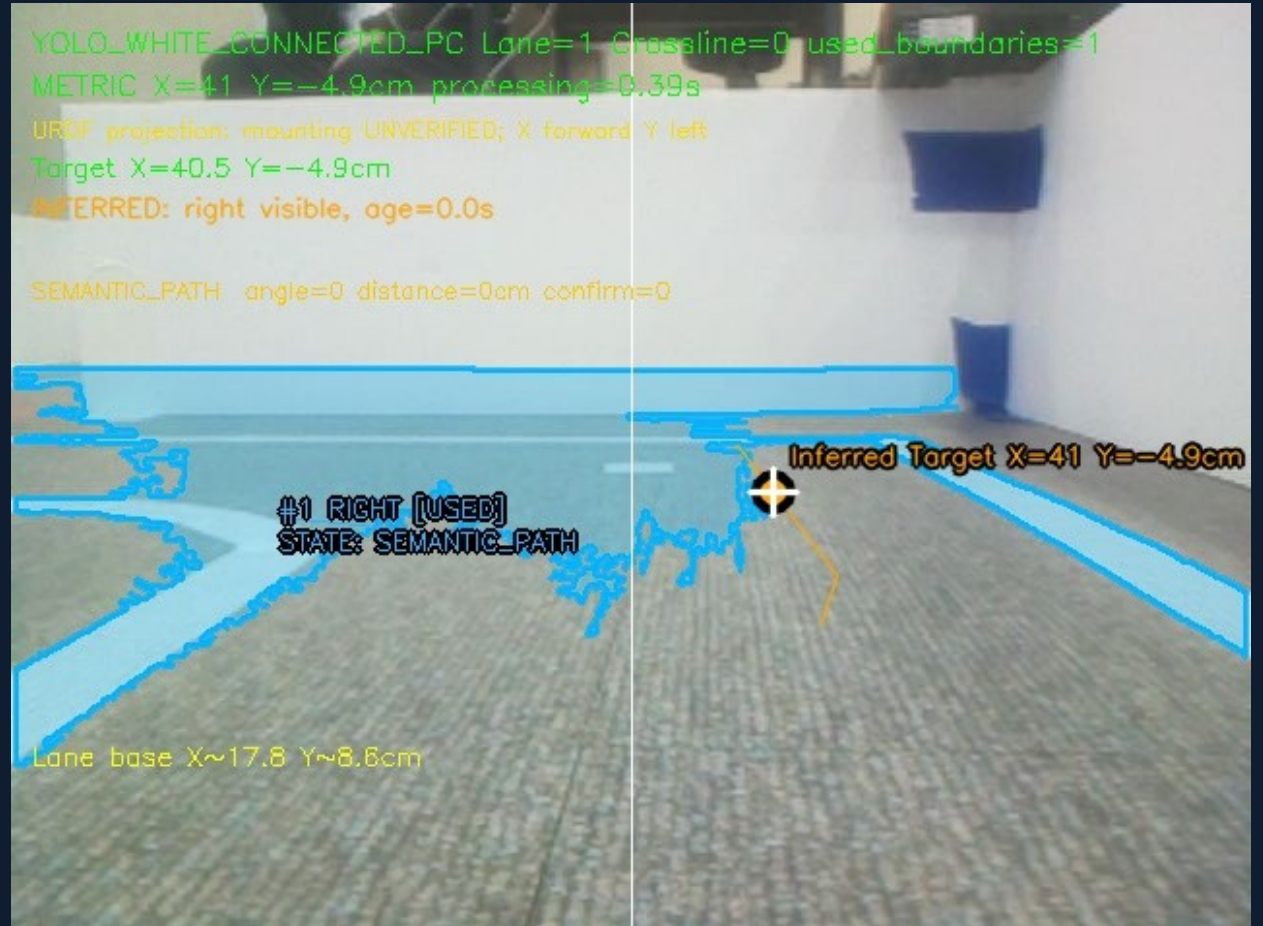
최신 관측이 무엇인지와 저장 경계로 움직일지는 별개
어느 모드에서 어떤 기억이 쓰이는지 확인해야 함

local_map 의 저장 · 표시 규칙

- base 관측 → 캡처 당시 odom 좌표로 변환해 기록
- SpatialLaneArchive: 1cm 셀, $\pm 1\text{m}$ 밖 관측 제거
 - 최근 이력 : 최대 600 프레임 / 10 초
 - 화면 : 480×480 , 1px=5mm, 그리드 10cm

디버그 화면에서 읽어야 할 정보

- Lane / used_boundaries: 검출 수와 실제 사용 수
- LEFT·RIGHT / USED: 선택한 경계의 역할
- Target X,Y: base 기준 실제 목표 좌표 (cm)
- SEMANTIC_PATH + timing + supplement 상태



정지 조건 : 신선도와 경로 유효성

- 현재 입력 또는 응답이 오래됨 : `result_timeout=0.8` 초
- fresh odometry/ 캡처 시점 pose 가 없으면 계획 거부
- 목표 경로가 없으면 `WAITING_FOR_LANE / zero Twist`
- 검출 정지선 · 주행 비활성화 · 시간 제한도 정지 원인

이번 실제 주행 : 완주가 아니라 실패 관측

- semantic=true / PC CPU / 최대 300초 제한
 - 시작 후 odom 기준 약 24cm 이동하고 정지
- path_error: no near white boundary
 - 경로 없음을 확인해 서비스로 비활성화 · zero 발행



검증 결과 : 시물과 실제를 분리

- 기존 성공 checkpoint: 90 도 2 회 + S 자 4 회 = 6/6
- 실차 포팅 코드 재검증 : 4/5 성공 , S 자는 2/3
- 패키지 회귀 테스트 : 673 개 통과 / 실패 0
- 이번 실제 로봇 : 경로 유실 정지 , 완주 미검증

실행 · 미리보기 · 정지 방법

- 로봇 : `ros2 launch pinky_move lane_autonomy_pinky2.launch.py`
- PC: `ros2 launch pinky_move lane_inference_pc.launch.py`
 - 미리보기 : `cam_view.py --topic /lane_autonomy/debug_image --transport raw`
 - 출발 전 확인 후 `enable` 서비스 `true` / 정지는 `false`

AprilTag 큐브 : 다음 확장 , 아직 주행 연동 아님

- YOLO: 차선 형상 / AprilTag: 코너 · 분기 식별
 - 고유 ID + 코너 의미 + 회전 시작점 관계를 등록
 - 여러 큐브가 보여도 로봇별 임무 순서로 선택
 - 기존 cube calibration $\neq$ runtime 태그 임무 주행

관제 + YOLO: Nav2 없는 임무 전달 설계

- 관제 52 → 로봇 20/22: mission_id · 태그 순서 · 시작 / 취소
- 로봇 → 관제 : ACK · 주행 중 · 도착 · 실패 · heartbeat
- 토픽 브릿지로 구현 가능 / 중복 · 유효기간 · 재시도 필요
- 각 로봇 /cmd_vel 발행자는 하나만 유지

두 로봇 순차 출발 : 도착 확인이 핵심

- 관제 : A 주행 허가 / B 대기 → A 도착 확인 → B 출발
- 태그가 보임 $\neq$ 빈 장소에 도착함
- 도착 조건 : 해당 ID + 근접 위치 + 정지 상태 + 일정 유지
- ACK 누락 · 통신 단절 · 실패 시 B 는 계속 대기

개선 순서 : 현재 실패를 해결하려면

- 1. 실측으로 projection 오차 · 태그 pose 를 검증
- 2. S 자 · 90 도 데이터에서 mask/ 연결 경계 실패를 재현
- 3. semantic 모드에 맵 보조를 넣으려면 별도 검증
- 4. 단독 주행 안정화 후 태그 임무 · 두 로봇 관제 연결

근거 파일과 현재 상태

- 코드 : ~/ws/pinky_pro/pinky_move/pinky_move/
- 설정 : config/lane_autonomy.yaml + robot_floor_calibration.json
- 검증 : hardware_semantic_port/report.html + summary.json
- 실제 영상 · 로그 : Downloads/cam/drive_20261006_ZxWSHs/
- 현재 : 제가 켜 로봇 노드 · PC 추론은 종료된 상태